# Instalación desde cero de la cadena completa MUSIC → IP-Glasma/3dMCGlauber → iS3D → UrQMD → CRAB3/be

Checklist con bugs corregidos, para un checkout nuevo en `xook`

Bi+Bi correlations campaign

2026-09-30

## 1. Alcance

Esta guía consolida en una sola lista, en orden de ejecución, todos los bugs de instalación/ejecución ya encontrados y corregidos a lo largo de las campañas Bi+Bi 5.8/7.7/9 GeV y AuAu 130/200 GeV (fuente: `music-install/README.md` §8, y los README de `music/BiBi_cascade_*GeV/`), pensada para levantar la cadena completa en una máquina nueva (`xook`) sin redescubrir cada uno por separado. Cierra con un chequeo de sanidad end-to-end: reproducir el *yield* de referencia Bi+Bi $\sqrt{s_{NN}} = 5.8$ GeV medido con una cascada UrQMD standalone, que es el número contra el que se calibra toda la cadena hidro en esa energía.

No repite el contenido físico/metodológico de los status reports ya existentes (`status_report_bibi58_cascade.t` `status_urqmd_crab3.tex`) — esta es una guía operativa de instalación.

## 2. Prerrequisitos del sistema

| Paquete | Para qué | Instalación (Ubuntu/Debian) |
|---|---|---|
| `g++` / `gfortran` | compilar MUSIC (C++) y UrQMD (Fortran legacy) | incluidos en `build-essential` + `gfortran` |
| `cmake` | build de MUSIC | `sudo apt install cmake` |
| `git` | clonar los repos | `sudo apt install git` |
| `libgsl-dev` | interpolación (MUSIC) | `sudo apt install libgsl-dev` |
| Python 3 + `numpy/numba/matplotlib` | yields, `be/` (C2/C3), plots | `sudo apt install python3 python3-numpy python3-matplotlib`, `pip install numba` |

**Ojo con `libgsl-dev` específicamente**: si solo está la librería en tiempo de ejecución (`libgsl28`) pero no los headers, `cmake` compila MUSIC igual pero *sin GSL y sin avisar con un error fuerte* — el binario queda incompleto y falla de formas confusas más adelante. Verificar esto explícitamente antes de compilar (o dejar que `install_music.sh` lo haga).

## 3. MUSIC (core)

1. Clonar e instalar:

```
git clone https://github.com/isadoji/music-install.git
cd music-install
./install_music.sh /software/mexnica/MUSIC/music
export MUSIC_DIR=/software/mexnica/MUSIC/music # agregar a .bashrc
```

2. **Aplicar parches conocidos antes de dar por buena la instalación**

   ```
   ./apply_music_fixes.sh /software/mexnica/MUSIC/music
   ```

   (`patches/`,este repo):

   - `freeze_pseudo.patch` — evita que `free(particleList)` cuelgue en corridas grandes de `freeze_pseudo.cpp` (bug de limpieza de memoria de MUSIC upstream al final del freeze-out).
   - `read_in_parameters.patch` — necesario si se va a usar `EOS_to_use 20` (2DTExS): el `read_in_parameters.cpp` de MUSIC stock solo acepta EOS $\leq 14$ de fábrica y rechaza el input sin este parche.

3. **2DTExS (`EOS_to_use 20`)** — EOS custom, ya vendorizada en este repo (`patches/2dtexs/`): archivos nuevos `eos_2dtexs.h`/`eos_2dtexs.cpp`, los reemplazos completos `eos.cpp`/`CMakeLists.txt` (los stock de MUSIC no registran la EOS 20) y la tabla `EoS2DTExS.dat` (41 MB, checksum verificado contra el original de `epos/src/MSt/`). Ya no depende de un checkout aparte de `epos/` al lado. Instalar con:

   ```
   ./apply_2dtexs.sh /software/mexnica/MUSIC/music
   ```

   Este script hace todo en un solo paso: copia los archivos nuevos, reemplaza `eos.cpp`/`CMakeLists.txt`, aplica `freeze_pseudo.patch` y `read_in_parameters.patch` (si ya se corrió `apply_music_fixes.sh` antes, los detecta ya aplicados y no falla), copia la tabla a `$MUSIC_DIR/EOS/2DTExS/` y recompila. Si solo se necesita la física estándar (`EOS_to_use 9`, hotQCD) no hace falta correr este paso — basta con `apply_music_fixes.sh` del punto anterior.

4. Gotchas menores ya manejados automáticamente por los scripts de `music-install` (no requieren acción, se listan para referencia): directorio de `--output` de SLURM debe existir antes de `sbatch` (`mkdir -p runs/<nombre>`); `min_pt 0.01` en vez de `0` exacto (evita el crash de GSL `x values must be strictly increasing`); `./outputs` se crea antes de modo 3; symlinks de `EOS//tables/` en el directorio de la corrida.

## 4. Condiciones iniciales — IP-Glasma / 3dMCGlauber

Para Bi+Bi a energías bajas (NICA/BES: 5.8, 7.7, 9 GeV) usar **3dMCGlauber** (`Initial_profile 13`), no IP-Glasma (que no tiene descripción nativa de bariones netos y no aplica a estas energías).

1. Instalar y parchar el núcleo de Bi-209 (no viene de fábrica):

```
./install_3dmcglauber.sh /software/mexnica/MUSIC/3dMCGlauber
                            # aplica patches/3dmcglauber_add_bi_nucleus.patch
export GLAUBER_DIR=/software/mexnica/MUSIC/3dMCGlauber
```

El parche agrega la parametrización de Bi-209 ($R = 6.96$ fm, $a = 0.537$ fm, convención arXiv:2401.00619).

2. **Recortar la salida a 25 columnas** antes de pasarla a MUSIC: la versión instalada de 3dMC-Glauber escribe 30 columnas (`strings_event_N.dat`, incluye campos de transporte de carga eléctrica que MUSIC `public_stable` todavía no lee). Sin el recorte, MUSIC aborta con `the format of file...is wrong`. Usar `generate_bibi_ic.sh` (ya hace el recorte), no la salida cruda de `3dMCGlb.e` directamente.

3. `Include_Rhob_Yes_1_No_0 1` es **obligatorio** a estas energías (frenado bariónico grande, $\mu_B$ finito), junto con una EOS que dependa de $\mu_B$ (`EOS_to_use 14`, `neos_bqs`, o `EOS_to_use 20` si ya se aplicó el parche de 2DTExS de la sección anterior) — nunca combinar el flag con una EOS a $\mu_B = 0$ (`EOS_to_use 2`/hotQCD), sería inconsistente.

4. Si se usa IP-Glasma en algún punto de comparación (p. ej. el benchmark a 200 GeV): confirmar que existe `nucleusConfigurations/<núcleo>.bin.in` *antes* de correr — su ausencia no da un error claro, produce un **loop infinito** que parece un OOM (consume 24–64–195 GB sin terminar). Con el archivo presente el uso real es ∼1.2 GB, ∼8 min. Además fijar `maxtime 0.4` explícitamente (`0.6` genera `epsilon-u-Hydro-t0.6-0.dat`, no el `t0.4` que esperan los scripts downstream); builds recientes además renombraron la salida a `epsilon-u-Hydro-TauHydro-0.dat`.

## 5. iS3D — muestreo de partículas sobre la superficie de freeze-out

1. Clonar, parchar y compilar con

   ```
   ./install_is3d.sh /software/mexnica/MUSIC/iS3D
   ```

   (este repo) — ya aplica el parche crítico de abajo automáticamente.

2. **Aplicar el parche crítico de `deltafReader.cpp` antes de usarlo para física con bariones.** Bug: `Deltaf_Data::calculate_ bilinear` indexa la tabla $\delta f$ como `f_data[iT][imuB]`, pero el loader la guarda como `f_data[iB][iT]` (ejes transpuestos). Con `include_baryon 1`, *todo* coeficiente $\delta f$ (F, G, $\beta_{\rm bulk}$, $\beta_V$, $\beta_\pi$) se lee de la celda equivocada —puede además leer fuera de rango si `iT>80`. Síntoma: $\bar{p}/p$, $\bar{n}/n$, $\bar{\Lambda}/\Lambda$ muy asimétricos incluso a $\mu_B \approx 0$ ($\bar{p}/p \sim 2.4$ en vez de $\sim 1$), K/$\pi$ inflado, bariones sistemáticamente bajos. El camino `include_baryon 0` (spline cúbico) **no** está afectado.

   - **Fix**: parchar `deltafReader.cpp` para indexar `[iB][iT]` correctamente y recompilar — `patches/iS3D_deltafReader_bilinear_fix.patch` en este repo, aplicado automáticamente por `install_is3d.sh`. **Verificar explícitamente que el checkout nuevo en `xook` lo traiga**, no asumirlo por venir del mismo repo.
   - Workaround si no se puede recompilar de inmediato: `include_baryon 0` en `iS3D_parameters.dat` (fuerza el camino spline) — solo válido si la superficie tiene $\mu_B \approx 0$.

3. Si la superficie viene de una corrida **boost-invariant** (2D), fijar `mode 8` explícitamente en la config de iS3D — el reader por defecto da resultados raros con superficies 2D. Confirmar con $\langle T \rangle / \langle \mu_B \rangle$ físicamente razonables tras correr.

## 6. UrQMD — afterburner / cascada

1. **Parche de compilación obligatorio con gfortran ≥10**: UrQMD 3.4 es Fortran legacy; gfortran ≥10 endureció el chequeo de tipos por defecto y falla con `Type mismatch/Rank mismatch`.

   `./install_urqmd.sh /software/mexnica/MUSIC/urqmd-3.4`

   (este repo) descarga el tar oficial de `urqmd.org` y aplica `patches/urqmd_Linux.mk.patch` automáticamente — agrega `-std=legacy -fallow-argument-mismatch -ffixed-line-length-none` antes de compilar.

2. **Semántica invertida de las banderas de salida**: listar una unidad de salida (`fXX`) en la tarjeta de entrada de UrQMD *suprime* esa salida (`bfXX=.true.` → no escribe), no la habilita. Para quedarse solo con `f19` (OSCAR1997A nativo, el formato correcto — **no** `f14`, que es el formato extendido propio de UrQMD y *no* es OSCAR), listar las que *no* se quieren: `f13 f14 f15 f16 f18 f20`, dejando `f19` sin listar.

3. Si se usa el wrapper del afterburner (no la cascada standalone): confirmar que siembra `frrx/frry/frrz/frr0` (posiciones reales de freeze-out) en el loop de `getoldevent` (`output.f`) — un wrapper viejo sin este parche deja esas posiciones en cero/no inicializadas, invalidando cualquier análisis de $c\tau$/core-halo. El parche está en `AuAu_200GeV/crab_analysis/patches/urqmd_afterburner_frzout_ creation_point.` **no** se retro-aplica a output ya generado antes del fix.

4. Usar siempre `f19`/`f20` nativos para cualquier análisis que dependa de la posición real de freeze-out — `f20` (`osc99_event`) escribe `frrx/frry/frrz/frr0` (coordenadas reales); `f14` (`file14out`) escribe `r0/rx/ry/rz` (posiciones *free-streamed*, distinta cantidad física) — confirmado leyendo `output.f` directamente.

5. `stop 137` **no es OOM**: UrQMD 3.4 puede abortar con `stop 137` desde `anndec.f:285` (canal de decaimiento sin probabilidades de rama, p. ej. `ityp 139`, $m = 2.112$ GeV) — es un exit code propio de UrQMD, no memoria. Si un job array pierde 3–43 % de tareas según config, revisar primero este crash antes de asumir OOM. Mitigación actual: correr en chunks pequeños (no hay reintento/skip automático implementado todavía).

## 7. CRAB3

1. Clonar, parchar y compilar con

   `./install_crab3.sh /software/mexnica/MUSIC/CRAB3`

   (este repo; CRAB3 no usa `cmake`, se compila directo con `g++ crab.cpp -o crab.e`). Aplica dos parches:

   - `Increase NPHASEMAX in crab.cpp`: el `#define NPHASEMAX 20000000` de `crab.cpp` es un arreglo *estático* de punteros (`NBMAX=1`) que guarda todos los $\pi^+$ de todos los eventos pooled del run; campañas grandes (∼1M eventos) lo superan y `crab_prinput.cpp` aborta. Fix vendorizado aquí: `patches/crab3_increase_nphasemax.patch` simplemente sube la constante a `200000000` — al ser un arreglo de punteros (no de objetos), el costo real de memoria (BSS) crece solo con las entradas realmente usadas, no con el tamaño declarado.
   - `patches/crab3_nmax_mixing_heap_fix.patch`: mueve los arreglos de `MIXED_PAIRS_FOR_DENOM` (`NMAX_FOR_MIXING=200000`) de la pila al heap en `crab_main.cpp` — evita overflow de stack.

2. Crear `results/` antes de correr `crab.e` — `fopen()` sin ese directorio segfaultea al arrancar.

3. **`--mem` de SLURM no se puede fijar estático**: la multiplicidad real ($\pi^+$/evento) solo se conoce corriendo el afterburner primero. Insertar un job intermedio "autosize" (muestrea un shard real ya escrito, proyecta el total, aplica margen $\sim$30 %) entre el afterburner y el job real de CRAB3 — patrón ya implementado en `music/BiBi_cascade_*GeV/figA3/slurm_*_crab3_autosize.sh`.

4. Si se actualiza memoria de un job ya enviado con `scontrol update JobId=<id>MinMemoryNode=<valor>`: el campo espera un entero plano en MB, *no* un sufijo `G` (`MinMemoryNode=32768`, no `32G`).

5. Antes de comparar contra un `crab3_qinv*.dat` de referencia: leer siempre el header — algunos usan `k=(p2-p1)/2:` (momento reducido) y otros `k=(p2-p1):` (Qinv completo); **no** es una convención fija por proyecto, cambia por build/config. Un factor $\sim$2 de diferencia en $R$ ajustado suele venir de esto.

## 8. `be/` — C2/C3 en Python

1. **Unidades**: `be/kernels.py` trabaja en GeV/fm con momento reducido $k = Q_{\rm inv}/2$. Los wrappers de alto nivel (`compute_c2_crab`, `compute_c2_batched`) convierten `maxmom_mev/1000` internamente — un driver que llama al kernel `numba` directo **no** hace esa conversión, y el fallo es silencioso (exit 0, arrays con la forma correcta, contenido mal: eje 0–50 GeV en vez de 0–50 MeV, $C_2 \approx 1.000$ plano). Convertir siempre `maxmom_gev = maxmom_mev / 1000.0` antes de binear si se llama `kernels.py` directo; validar contra el estimador pool-random como cross-check.

2. **Simetrización de Bose–Einstein en C3**: `weight = w12*w13*w23` (producto de pesos pairwise) **no** es la simetrización genuina para $N = 3$ bosones idénticos — hay que sumar sobre las $3! = 6$ permutaciones de $S_3$. Usar `kernels.triplet_be_weight()` (ya suma las 6 fases de permutación directo de los cuadrivectores).

3. **Memoria**: la implementación original de `c2.py`/ `c3.py` retenía cada par/triplet muestreado en memoria (RSS escala linealmente: 100M triplets $\rightarrow$ 24 GB, 400M $\rightarrow$ 88 GB; `n_pairs=2e8` $\rightarrow \sim$30 GB). Usar el refactor de reservoir acotado (`reservoir_size`, ya presente) — RSS acotado independiente de `n_pairs`/`n_triplets`; correr además bajo `ulimit -v` como cinturón de seguridad en jobs grandes.

4. El reservoir del denominador mixto en `compute_c3_diagonal_crab` debe llenarse **sin** condición desde todo intento crudo, no solo con triplets ya aceptados por el corte diagonal — llenarlo condicionado sesga la referencia mixta hacia abajo (meseta sospechosa $\sim$0.5–0.6 a $k$ grande).

5. `plotting.py`'s `plot_c2()`: enmascarar `ref_err >= 0` antes de graficar (algunos `.dat` usan el sentinel $-1$ para "sin datos" en un bin; sin la máscara, `yerr` negativo crashea el errorbar).

## 9. Gotchas generales de SLURM

- **Nunca fijar `--time`** en estos jobs — dejar el default del cluster (sin límite); un `--time` viejo ya causó pérdida de 20/60 tareas por `TIMEOUT` en la campaña de cascada UrQMD standalone (§11).

- **`$0`/`dirname "$0"` no localiza el propio script bajo un job array**: SLURM copia el script a un spool dir (`/var/lib/slurm/slurmd/jobNNN/`) y ejecuta la copia — resolver rutas relativas a scripts "vecinos" desde `$SLURM_SUBMIT_DIR` o hardcodear la ruta absoluta, no derivarla en runtime.

- Un `glob` de muchos archivos de shards falla con `Argument list too long` (excede `ARG_MAX`) — usar `find ... -exec` o iterar desde una lista en archivo.
- `acaxees02` no monta `/storage` de forma consistente — confirmar el mount antes de lanzar un array que dependa de él.
- Los nodos del cluster pueden tener velocidad desigual — no asumir que todas las tareas de un array terminan en tiempo similar.

# 10. Configurar acceso sin password (para lanzar/monitorear sin dejar la terminal abierta)

Si `xook` necesita transferir datos hacia/desde otro nodo del cluster por `rsync`/`ssh` de forma no interactiva (jobs largos en `screen`/`nohup`), generar una llave dedicada en vez de depender de password:

```
ssh-keygen -t ed25519 -f ~/.ssh/xook_<destino> -N ""
ssh-copy-id -i ~/.ssh/xook_<destino>.pub usuario@destino
```

y usarla explícitamente en el `rsync`: `-e "ssh -i  /.ssh/xook_<destino>"`. Lanzar transferencias largas con `screen -dmS <nombre>bash script.sh` para poder desengancharse (`Ctrl+A` luego `D`, **no** `Ctrl+D`) sin matar el proceso.

# 11. Validación: yield Bi+Bi $\sqrt{s_{NN}} = 5.8$ GeV vs. cascada UrQMD standalone

Antes de confiar en la cadena hidro completa (MUSIC → iS3D → UrQMD → CRAB3) recién instalada en `xook`, se recomienda reproducir el número de referencia contra el que se calibra `s_factor`/los parámetros de `3dMCGlauber` a esta energía: el yield final de $\pi^+$ de una cascada UrQMD **standalone** (sin MUSIC, sin iS3D — solo UrQMD puro), que sirve de "blanco" independiente de cualquier suposición hidrodinámica.

## 11.1. Instalación de la cascada standalone

Binario distinto al afterburner del pipeline hidro (`urqmd.x86_64` vs. `urqmd_afterburner`) — mismo repo `urqmd-3.4`, mismo parche de compilación de la §6 (`Linux.mk.patch`, obligatorio con gfortran $\geq$10), pero compilado en modo standalone (genera su propia colisión núcleo-núcleo en vez de leer una superficie de freeze-out de iS3D). Instalar con `./install_urqmd.sh` (este repo).

## 11.2. Tarjeta de entrada

Reconstruida leyendo `input.f` (sin documentación externa disponible):

```
pro 209 83        # proyectil: Bi (A=209, Z=83)
tar 209 83        # blanco:    Bi (A=209, Z=83)

nev <N>           # numero de eventos
IMP 0.0 1.0       # parametro de impacto explicito (fm), central
```

```
ecm 5.8           # sqrt(s_NN) por par NN (srtflag=1)
eos 0             # 0 = cascada pura, sin potenciales
rsd <seed>
tim 200 200       # tiempo total de propagacion / intervalo de salida (fm/c)

f13
f14
#f19              # NO listar: es la salida OSCAR1997A real que se necesita
f15
f16
f20               # listar = suprimir esa unidad (ver bug de semantica, S5)

xxx               # fin de tarjeta
```

Ejecución: variables de entorno `ftn09`=archivo de entrada, `ftn13..ftn20`=archivos de salida por unidad Fortran, luego `./urqmd.x86_64`.

**Gotcha ya encontrado en esta misma validación**: el piloto original se corrió creyendo que `f14` era el formato OSCAR1997A — es incorrecto (`f14` es el formato extendido nativo de UrQMD, columnas `t X Y Z E Px Py Pz m ityp iso chg lcl# ncl or`). `f19` es el OSCAR1997A verdadero (con PDG ids vía `pdgid()`). Confirmar que la tarjeta de entrada deja `f19` habilitado (sin listar) antes de lanzar la campaña completa, no solo un evento de prueba.

### 11.3. Paralelizar en SLURM

Job array, cada tarea en un directorio aislado (`mktemp -d`) para evitar condiciones de carrera al escribir archivos de nombre fijo:

```
sbatch --array=0-59 scripts/slurm_urqmd_cascade_array.sh \
    <n_total_events> <outdir> <base_seed>
```

Sin `--time` (§9) — un límite de tiempo viejo hizo perder 20/60 tareas por `TIMEOUT` en la corrida de referencia original.

### 11.4. Cálculo del yield y comparación con el blanco

Sobre el `f19` combinado (OSCAR1997A, filtrando solo $\pi^+$), calcular la rapidez real por partícula,

$$y = \tfrac{1}{2}\ln\!\left(\frac{E + p_z}{E - p_z}\right),$$

y contar $\pi^+$ con $|y| < 0.5$:

$$\left.\frac{dN_{\pi^+}}{dy}\right|_{|y|<0.5} = \frac{N_{\pi^+}(|y| < 0.5)}{N_{\text{eventos}}}.$$

**Valor de referencia ya medido** (job 12333, 66,670 eventos sobrevivientes de 100,000 lanzados — 20/60 tareas se perdieron por el `TIMEOUT` descrito arriba, no relanzadas):

$$\left.\frac{dN_{\pi^+}}{dy}\right|_{|y|<0.5} = \frac{5{,}717{,}085}{66{,}670} = 85.75.$$

**Criterio de aceptación para el checkout nuevo en `xook`**: correr el mismo número de eventos (o una fracción estadísticamente suficiente, $\gtrsim$10,000) con la misma tarjeta de entrada (`ecm 5.8`, `IMP 0.0 1.0`, `eos 0`) y confirmar que el $dN_{\pi^+}/dy(|y| < 0.5)$ medido cae dentro del error estadístico de

85.75. Una discrepancia grande indica que algún parche de la §6 (semántica de banderas `fXX`, o el parche de compilación de gfortran) no se aplicó correctamente en la instalación nueva, **antes** de invertir tiempo de cómputo en la cadena hidro completa.

Este mismo yield (85.75) es además el "blanco de calibración" usado después para ajustar `3dMCGlauber/s_factor` en las corridas hidro completas a esta energía (ver `BiBi_cascade_5.8GeV/README.md`, secciones de escaneo de `s_factor` y barrido de `shadowing_factor/BG`) — reproducirlo aquí no es solo un chequeo de instalación, es el primer paso real de esa calibración.

## 12. Resumen: orden de instalación recomendado

1. Prerrequisitos del sistema (§2).
2. MUSIC + parches `freeze_pseudo/read_in_parameters` (§3).
3. 3dMCGlauber + parche de núcleo Bi-209 (§4).
4. UrQMD (standalone $y$ afterburner) + parche de compilación gfortran (§6, §11).
5. **Validación de yield 5.8 GeV vs. cascada UrQMD** (§11) — antes de seguir.
6. iS3D + parche crítico `deltafReader.cpp` (§5).
7. CRAB3 (§7).
8. `be/` (§8), solo si se van a correr correlaciones C2/C3 además del pipeline de yields/espectros.